# LABORATORIO DI INGEGNERIA DEI SISTEMI SOFTWARE

## Introduction

## Requirements

Costruire un sistema software distribuito costituio da N (N>=1) Produttori che inviano informazione a 1 Consumatore, il quale deve elaborare tale informazione. La dislocazione dei componenti sui nodi di elaborazione può essere: OneNode: tutti i componenti operano nello stesso nodo; TwoNodes: gli N Produttori operano in uno stesso nodo, mentre il Consumatore opera in un diverso nodo; ManyNodes: il Consumatore opera in suo proprio nodo, mentre i Produttori operano su K nodi diversi (1 < K <=N).

## Requirement analysis

- R1: I produttori inviano informazione al consumatore che la elabora
- R2: n. produttori >= 1
- R3: n. consumatori = 1
- R4: modalità OneNode := produttori e consumatore sullo stesso nodo
- R5: modalità TwoNodes := 1 nodo per consumatore + 1 nodo per tutti i produttori
- R6: modalità ManyNodes := 1 nodo per consumatore + K nodi per i produttori (1 < K <=N)

## Problem analysis

Da R1,R2 ed R3 intuiamo che il sistema è composto da due tipologie di entità ovvero Produttore e Consumatore, ci sarà almeno un Produttore ed esattamente un Consumatore.
I Produttori devono essere in grado di comunicare con il Consumatore, indipendentemente che questi si trovi sul loro stesso nodo (R4) o su un altro nodo (R5,R6).
Da R2 ed R3 risultano applicabili diversi modelli, in particolare i due che sembrano più adatti sono:

- il modello Cliente/Servitore in cui il produttore è un cliente ed il consumatore è un servitore
- il modello ad attori, nel quale entrambe le entità sono rappresentate da attori che comunicano attraverso messaggi

a seguito dell'analisi svolta in aula si sceglie di proseguire con il modello ad attori.

## Test plans

per questo sprint si vuole testare principalmente la parte relativa alla comunicazione OneNode utilizzando il modello ad attori, e nello specifico il Qak. per questo motivo i test previsti sono due:

- **TestDispatch**: come Producer, mi aspetto: di inviare un dispatch cone payload infoDis(10) e che il Consumer la elabori.
- **TestRequest**: come Producer, mi aspetto:
  1. di inviare una request in modo sincrono cone payload infoReq(11) e di verificare che il Consumer mi invii la risposta infoRepl(11);
  2. che altri Producer possano agire allo stesso modo
  3. e che il Consumer non mi invii la risposta destinata a un altro;
  4. di inviare una request R1 in modo asincrono, di eseguire altre azioni e di poter ricevere la risposta in un secondo momento
  5. Se le mie altre azioni includono una nuova richiesta R2 sincrona, la risposta non dovrebbe essere quella relativa a R1. L'osservabilità deriva qui dalla possibilità di ricevere risposta dal Consumer.

## Project

in questo sprint si vuole implementare una versione del sistema utilizzando il linguaggio Qak. trattandosi di una prima implementazione in questo linguaggio si opta per mantenere l'intero sistema su un singolo nodo.
si sceglie dunque di rappresentare Prod e Cons come due attori a stati finiti.

- sono stati creati due attori di tipo produttore **Prod0** e **Prod1** con i seguenti stati:
  - **s0**: stato iniziale, l'attore notifica di aver iniziato ad eseguire, in seguito effettua una transizione spontanea verso lo stato sendInfo
  - **sendInfo**: l'attore invia due messaggi in sequenza al consumatore, il primo di tipo dispatch ed il secondo di tipo request, all'arrivo della risposta al secondo messaggio l'attore transita allo stato replReceived
  - **replReceived**: l'attore notifica di aver ricevuto risposta al messggio inviato e transita spontaneamente allo stato wait
  - **wait**: l'attore attende 5 secondi e transita spontaneamente allo stato s0

- **Cons** è stato progettato con i seguenti stati:
  - **s0**: stato iniziale, l'attore notifica di aver iniziato ad eseguire, in seguito effettua una transizione spontanea verso lo stato listen
  - **listen**: l'attore resta in attesa di messaggi, all'arrivo di un messaggio transita verso lo stato handleInfoReq o handleInfoDis a seconda che il messaggio sia di tipo richiesta o dispatch
  - **handleInfoDis**: l'attore notifica di aver ricevuto un messaggio di tipo dispatch, ne stampa il contenuto e transita spontaneamente verso lo stato di listen
  - **handleInfoReq**: l'attore notifica di aver ricevuto un messaggio di tipo request, ne stampa il contenuto, invia una risposta al mittente e transita spontaneamente verso lo stato di listen

## Testing

- **TestDispatch** è stato superato correttamente
- **TestRequest** è stato superato parzialmente, nello specifico i punti dall' 1. al 3. sono stati superati, il punto 4. risulta facilmente risolvibile, per esempio introducendo una transizione spontanea da sendInfo ad un nuovo stato moreAction in cui vengono eseguite nuove azioni prima di recuperare il messaggio, allo stesso modo anche il punto 5. risulta facilmente risolvibile per esempio introducendo nuovi tipi di request con le relative reply e modificando le transizioni in considerazione dell'ordine nel quale si vogliono ottenere le varie risposte.
  questi ultimi due punti non sono stati tuttavia risolti perchè l'implementazione dipende dal protocollo di comunicazione dal tipo di elaborazione sui quali non sono state fornite indicazioni specifiche

## Deployment

## Maintenance

By Student: Alessio Benenati
Email: alessio.benenati2@studio.unibo.it

GIT repo: https://github.com/alessioBenenatiUnibo/IngegneriaSistemiSW